

Credit Lecture S1: From a Fixed-Horizon PD to a Discrete Hazard

Deep Learning for Actuarial Modelling: a credit risk companion

AUTHOR

Mario Ervedosa

PUBLISHED

September 1, 2026

ABSTRACT

Credit lecture 1 measured default over a fixed outcome window of k months and set the second of its two exposure conventions aside, noting that months at risk is a discrete hazard and the door into survival analysis. This lecture walks through that door. It builds the survival table for the Bondora loan book, derives the survival product that turns a sequence of monthly hazards into a lifetime PD, and shows that a discrete-time hazard model is a logistic regression on a person-period expansion, so a lender's existing scorecard machinery already fits survival models. It closes by measuring what treating early repayment as censoring costs, which is 3 percentage points over twelve months and 17 over sixty, and hands the correction to lecture S2. This lecture answers no summer school lecture; it exists because credit lecture 1 opened a door.

1 Where lecture 1 stopped

Lecture 1 offered two ways to measure the exposure of a loan and took the first. Measuring exposure in windows gives every retained loan the same one, $v_i \equiv 1$ window of k months, and the frequency collapses to the default indicator $D_i^{(k)} = \mathbf{1}\{T_i \leq k\}$ itself. That is how application scorecards are built, and lecture 1 built one. Measuring exposure in months at risk instead gives each loan the time it actually spent exposed,

$$v_i = \min(T_i, k) \text{ months,}$$

and here the frequency does not collapse. A loan defaulting in month two and a loan defaulting in month eleven both score $D_i^{(k)} = 1$, yet they carry intensities differing by a factor of five, and the

second convention keeps the difference the first discards.

This lecture takes the second convention seriously. The quantity $\min(T_i, k)$ is the exposure of a **discrete hazard** model, and the machinery built on it answers questions the fixed-horizon PD cannot reach: when default arrives rather than whether it arrives inside one window, and what the PD is over any horizon rather than over the single horizon the flag was cut for. IFRS 9 needs exactly that, because an expected credit loss over the lifetime of an instrument needs a PD for every month of that lifetime.

The reader is assumed to hold lecture 1 and nothing else. Every symbol it fixed carries over unchanged, and the three time axes in particular: g_i for the vintage or origination month of loan i , t for months on book, which is the attained age of the loan, $u = g + t$ for calendar month, k for the horizon being asked about, and τ for the snapshot date, a single date across this extract, 20 July 2021. Borrower age is a covariate here and is written x_i , deliberately not standing in for the actuarial x , whose role t plays.

```
import numpy as np
import polars as pl
import pandas as pd
import matplotlib.pyplot as plt

plt.rcParams.update({
    "figure.figsize": (9, 4), "figure.dpi": 150, "font.size": 9,
    "axes.spines.top": False, "axes.spines.right": False,
})
OX, GREY, RULE, BLUE = "#8C2F1E", "#B0B0B0", "#444444", "#1F4E79"

surv = pl.read_parquet("../data/bondora_survival.parquet")
print(f"{surv.height:,} loans, {surv.width} columns")
```

179,235 loans, 54 columns

2 Two clocks, and two symbols that must stay apart

Lecture 1 defined $A_i = \tau - g_i$, the months of observation **available** on loan i at the snapshot, and used it to retain loans whose outcome window had closed, $\mathcal{W}_k = \{i : A_i \geq k\}$. The survival literature defines a different quantity with a confusingly similar role, the **censoring time** C_i in $\tilde{T}_i = \min(T_i, C_i)$, meaning the point at which the loan stopped being at risk.

However, these are not the same object, and conflating them biases every lifetime PD computed afterwards. Availability is a property of the calendar: it asks how long the snapshot could in principle have watched this loan. Censoring is a property of the loan: it asks when the loan actually left the risk set. A loan repaid in month nine left the risk set in month nine, whatever the calendar would have allowed.

WHY LECTURE 1 COULD GET AWAY WITH A_i AND THIS LECTURE CANNOT

Lecture 1's retention rule $A_i \geq k$ is a legitimate **calendar sample restriction**. It asks only whether a full window could have been observed, and it then reads the outcome off a complete window, so the answer it gets is right for the loans it keeps.

A survival likelihood asks a different question of every loan, namely how long it contributed to the risk set, and it needs the actual exit for that. In this extract the two answers diverge sharply. Of the 179,235 loans, 42,144 were repaid without ever defaulting, and those left the risk set at repayment, typically years before $\tau - g_i$ ran out.

```
gap = surv.filter(pl.col("exit_kind") == "settled").select(
    median_exit=pl.col("t_obs").median(),
    median_available=pl.col("A").median(),
)
print(gap)
print(f"\nmedian months at risk before settlement : {gap['median_exit'][0]:.1f}")
print(f"median months the calendar allowed      : {gap['median_available']
    [0]:.1f}")
```

shape: (1, 2)

median_exit	median_available
---	---
f64	f64
9.659138	37.256674

```
median months at risk before settlement : 9.7
median months the calendar allowed      : 37.3
```

The median settled loan left the risk set after about ten months, whereas the calendar would have permitted roughly three and a half years of observation. Using A_i as the censoring time would therefore credit those loans with years of exposure they never had, and would drive every estimated hazard downwards.

One further clash is worth declaring at the outset, before it causes confusion. Botha and Verster (2025) use τ with subscripts for the entry, resolution and stopping times of a spell, whereas lecture 1 fixed τ as the snapshot date. Both conventions appear in the credit survival literature. This lecture keeps lecture 1's meaning and writes exit times as durations in months on book, so τ never carries a subscript here.

3 The survival table

The modelling table lecture 1 used, `bondora_pd.parquet`, deliberately cannot serve here. It keeps only seasoned loans and records whether default fell inside one fixed window, whereas a survival model needs every loan and needs to know when each one left the risk set.

`scripts/convert_credit_data.py` therefore builds a second table, `bondora_survival.parquet`, over all 179,235 loans.

HOW EACH LOAN'S EXIT WAS DETERMINED

Botha and Verster (2025, Eq. 1) coalesce the ways a performing spell can resolve into a single nominal variable R_{ij} over four levels, accompanied by a censoring indicator $c_{ij} \in \{0, 1\}$ which is one for a right-censored spell:

$$R_{ij} = \begin{cases} 1 : \text{Default} & \text{if } c_{ij} = 0 \text{ and the default criteria apply} \\ 2 : \text{Settled} & \text{if } c_{ij} = 0 \text{ and the settlement criteria apply} \\ 3 : \text{Write-off / Other} & \text{if } c_{ij} = 0 \text{ and the write-off criteria apply} \\ 4 : \text{Censored} & \text{if } c_{ij} = 1. \end{cases}$$

Bondora's `Status` field maps onto three of these four, with write-off unobservable because the extract's write-off columns are post-origination and were excluded from the repository's tables. Each loan here contributes a single performing spell, so the spell index j is always one and is dropped from the notation.

The mapping needs one rule that is easy to get wrong. **A `DefaultDate` takes precedence over `Status`**, because default is absorbing. 10,743 loans carry a `Status` of Repaid and a `DefaultDate` together, having defaulted and later recovered, and reading `Status` first would score all of them as settlements and discard the event. The 52,887 loans marked Repaid therefore split into 10,743 defaults and 42,144 settlements.

Two approximations in the settlement timings are worth stating, since neither is visible in the counts. 855 settled loans carry a recorded end date falling after the snapshot, i.e. a scheduled date standing in for an unobserved early repayment, and the converter caps those at τ , which is the longest exposure consistent with the loan having ended by then. A further three carry an end date equal to their origination date, and the whole-month floor gives each of them one month of exposure it did not have. Both groups are small enough to leave every figure in this lecture unchanged, and both are the kind of thing that should be said out loud rather than absorbed silently.

```
print(surv.groupby("exit_kind").agg(
    loans=pl.len(),
    median_months=pl.col("t_obs").median().round(1),
).sort("loans", descending=True))
```

shape: (3, 3)

exit_kind	loans	median_months
---	---	---
str	u32	f64
default	71416	9.6
censored	65675	16.9
settled	42144	9.7

That gives 71,416 defaults, 42,144 settlements and 65,675 censored observations. The censored group holds the 57,611 loans still Current at the snapshot together with 8,064 loans that were Late without having reached Bondora's 60-days-past-due threshold, and those are therefore still at risk.

Two columns encode the outcome. The observed duration `t_obs` is $\tilde{T}_i = \min(T_i, C_i)$ in months, alongside `t_obs_m`, the same quantity in whole months and floored at one so that a loan exiting inside its first month still contributes that month to the risk set. The event indicator is $\delta_i = \mathbf{1}\{R_i = 1\}$, and it is derived from the exit kind rather than stored in its place, which matters: a table carrying only δ_i would have baked the treatment of early repayment irreversibly into the data, and the last section of this lecture measures what that treatment costs.

4 Survival, hazard, and why a lifetime PD is a product

Let T_i be the time in whole months from origination to default. In discrete time the three functions that describe it are the survival function, the marginal default probability and the hazard. Writing $t_{(k)}$ for the k th ordered month, Botha and Verster (2025, Eqs. 7 to 10) give the identities this lecture rests on:

$$f(t_{(k)}) = S(t_{(k-1)}) - S(t_{(k)}), \quad h(t_{(k)}) = 1 - \frac{S(t_{(k)})}{S(t_{(k-1)})}, \quad f(t_{(k)}) = S(t_{(k-1)}) \cdot h(t_{(k)}),$$

with $S(t_{(0)}) = 1$, since a loan cannot default before it exists. The hazard is the conditional default probability, meaning the proportion of the risk set at the start of month k that defaults during it. Inverting the middle identity and chaining it gives the result that reorganises how a credit modeller thinks about a lifetime PD:

$$S(t_{(k)}) = \prod_{s: t_{(s)} \leq t_{(k)}} (1 - h(t_{(s)})).$$

This lecture writes the individual conditional default probability as $q_{i,t}$ in place of h , following the notation of Mario's credit risk notation lecture and the actuarial q_x it descends from, and writes ${}_k p_{i,t}$ for the probability that loan i , currently t months on book, survives a further k months. That has the shape of ${}_t p_x$ with (k, t) in the roles of (t, x) . Everything in this lecture is measured from origination, so $t = 0$ throughout and the second subscript is dropped to keep the formulae readable. Botha's $h(t_{(k)})$ and the $q_{i,t}$ used here are the same object. In that notation,

$$q_{i,t} = \mathbb{P}(T_i = t \mid T_i \geq t, \mathbf{X}_i), \quad {}_k p_{i,0} = \prod_{t=1}^k (1 - q_{i,t}), \quad \text{PD}_k(\mathbf{X}_i) = 1 - {}_k p_{i,0}.$$

Setting $k = 12$ recovers exactly the quantity lecture 1 modelled, so the fixed-horizon PD is the special case of this expression at one chosen horizon. What the hazard formulation adds is every other horizon at no extra cost, which is the IFRS 9 term structure of default risk.

A LIFETIME PD IS NEVER A SUM OF MONTHLY PDS

The single most expensive arithmetic error in lifetime PD work is to add monthly hazards instead of chaining survival. Summing them ignores that a loan defaulting in month three is not available to default again in month four, so the sum double-counts and can exceed one. The Bondora hazards make the point without any need for a synthetic example, as the next section shows.

5 The life table and the Kaplan-Meier estimator

Estimating the hazards needs only two counts per month: the size n_k of the risk set at the start of month k , and the number d_k of defaults during it. Botha and Verster (2025, Eq. 11) note that setting $\hat{q}_t = d_t/n_t$ leads directly to the Kaplan-Meier estimator of Kaplan and Meier (1958). Both are computed here directly from the risk-set counts, with no survival package involved, which keeps the repository's `requirements.txt` untouched and makes the arithmetic visible.


```
HORIZON = 60 # months; the modal contractual term in this book is 60
```

```
def life_table(df, horizon=HORIZON):
    """Risk set, defaults and other exits per whole month on book."""
    t = df["t_obs_m"].to_numpy().clip(max=horizon + 1)
    kind = df["exit_kind"].to_numpy()
    months = np.arange(1, horizon + 1)
    n = np.array([(t >= m).sum() for m in months], dtype=float)
    d = np.array([(t == m) & (kind == "default").sum() for m in months],
                  dtype=float)
    w = np.array([(t == m) & (kind != "default").sum() for m in months],
                  dtype=float)
    return months, n, d, w

def kaplan_meier(n, d):
    """Return the discrete hazard, the KM survival curve and its Greenwood se."""
    q = np.divide(d, n, out=np.zeros_like(d), where=n > 0)
    S = np.cumprod(1.0 - q)
    with np.errstate(divide="ignore", invalid="ignore"):
        terms = np.where(n * (n - d) > 0, d / (n * (n - d)), 0.0)
    se = S * np.sqrt(np.cumsum(terms))
    return q, S, se

months, n_k, d_k, w_k = life_table(surv)
q_t, S_t, se_t = kaplan_meier(n_k, d_k)

table = pl.DataFrame({
    "month": months, "at risk": n_k.astype(int), "defaults": d_k.astype(int),
    "other exits": w_k.astype(int), "hazard": q_t.round(5),
    "S(t)": S_t.round(5), "PD_t": (1 - S_t).round(5),
})
print(table.filter(pl.col("month").is_in([1, 2, 3, 4, 6, 12, 24, 36, 48, 60])))
```

```
shape: (10, 7)
```

month	at risk	defaults	other exits	hazard	S(t)	PD_t
---	---	---	---	---	---	---
i64	i64	i64	i64	f64	f64	f64
1	179235	0	8601	0.0	1.0	0.0
2	170634	1	6493	0.00001	0.99999	0.00001
3	164140	258	6435	0.00157	0.99842	0.00158
4	157447	7240	4896	0.04598	0.95251	0.04749
6	132707	4685	4070	0.0353	0.86651	0.13349
12	89627	3203	2141	0.03574	0.69235	0.30765
24	35121	766	2059	0.02181	0.49019	0.50981
36	13652	187	780	0.0137	0.39354	0.60646
48	4824	73	243	0.01513	0.32439	0.67561

60	2061	16	114	0.00776	0.2832	0.7168
----	------	----	-----	---------	--------	--------

The first two rows carry a structural feature of the default definition rather than a quirk of the data. Bondora marks a loan defaulted at 60 days past due, so no loan can reach that threshold in its first month, and exactly one does in its second. The hazard is therefore genuinely zero at $t = 1$, which will matter when the model of the next section tries to estimate a parameter for it.

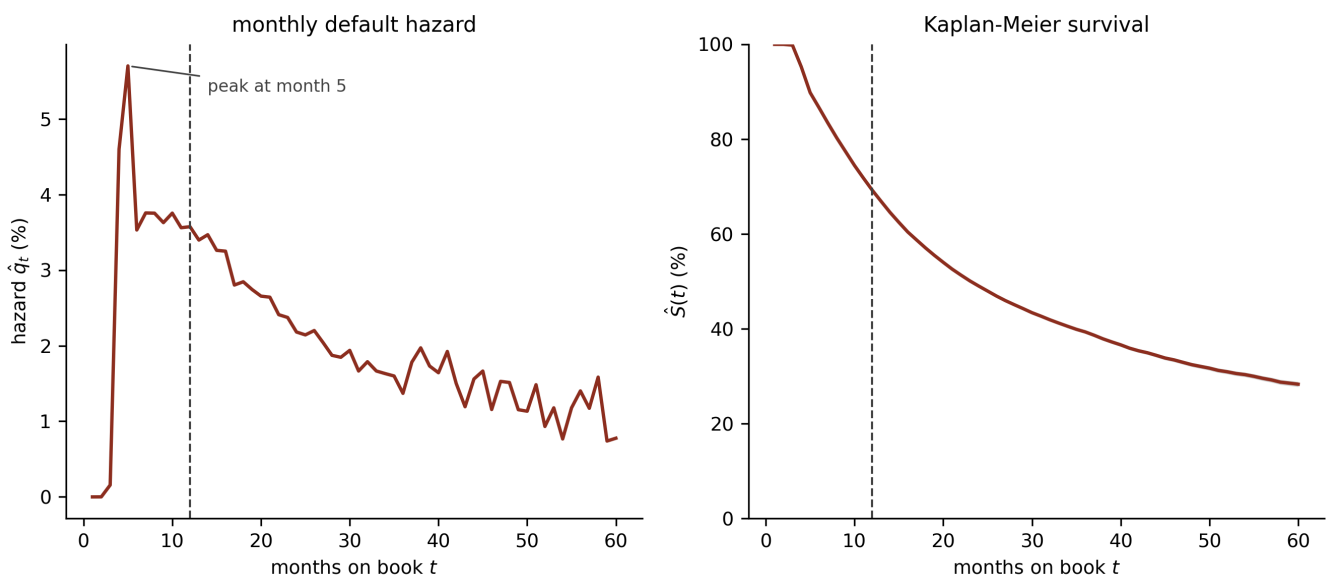
```

fig, axs = plt.subplots(1, 2)

ax = axs[0]
ax.plot(months, q_t * 100, color=0X, lw=1.6)
ax.axvline(12, color=RULE, ls="--", lw=1)
ax.set_xlabel("months on book $t$"); ax.set_ylabel("hazard $\hat{q}_t$ (%)")
ax.set_title("monthly default hazard", fontsize=10)
peak = int(months[np.argmax(q_t)])
ax.annotate(f"peak at month {peak}", xy=(peak, q_t.max() * 100),
           xytext=(peak + 9, q_t.max() * 100 * 0.94), fontsize=8, color=RULE,
           arrowprops=dict(arrowstyle="--", color=RULE, lw=0.8))

ax = axs[1]
ax.fill_between(months, (S_t - 1.96 * se_t) * 100, (S_t + 1.96 * se_t) * 100,
               color=GREY, alpha=0.55, lw=0)
ax.plot(months, S_t * 100, color=0X, lw=1.6)
ax.axvline(12, color=RULE, ls="--", lw=1)
ax.set_xlabel("months on book $t$"); ax.set_ylabel("$\hat{S}(t)$ (%)")
ax.set_ylim(0, 100)
ax.set_title("Kaplan-Meier survival", fontsize=10)
plt.tight_layout(); plt.show()

```



The Bondora loan book in discrete time, all 179,235 loans, with early repayment treated as censoring (the latent risks approach). (lhs): the estimated monthly default hazard $\hat{q}_t$, which peaks at 5.70 per cent in month five and then declines as the book seasons. (rhs): the Kaplan-Meier survival curve with a Greenwood 95 per cent interval, shown as a band; the interval is narrow enough over the first three years to be hard to separate from the curve. The dashed vertical marks the twelve-month horizon lecture 1 modelled.

The hazard curve is the **lifecycle** component of Breeden's (2016) age-period-cohort decomposition, which lecture 1 cited for the collinearity it forces. Here it is measured: default risk rises steeply to a peak in the fifth month on book, then falls away for the remaining fifty-five. A

fixed-horizon flag cut at twelve months averages over the whole of that shape and reports one number.

5.1 The sum against the product

With the hazards in hand the arithmetic error named above can be priced.

```
for k in (12, 24, 60):
    product = 1 - np.prod(1 - q_t[:k])
    naive = q_t[:k].sum()
    print(f"k = {k:2d} months    survival product {product:7.2%}    "
          f"sum of hazards {naive:7.2%}    overstatement {naive - product:+.2%}")
```

k = 12 months	survival product	30.76%	sum of hazards	36.01%	overstatement
					+5.24%
k = 24 months	survival product	50.98%	sum of hazards	70.04%	overstatement
					+19.06%
k = 60 months	survival product	71.68%	sum of hazards	124.46%	overstatement
					+52.78%

Over twelve months the sum overstates the PD by 5.2 percentage points, which a reviewer might absorb as a rounding difference. Over sixty months it returns 124.5 per cent, which is not a probability at all. The failure is not subtle at long horizons, and lifetime PD work lives at long horizons.

5.2 Stratifying the curve

A survival curve becomes a modelling tool once it is cut by a covariate. Bondora assigns each loan a `Rating` at listing time, which lecture 1 kept precisely so that modelling on top of another model's output could be discussed.

```

from scipy import stats

RATINGS = ["AA", "A", "B", "C", "D", "E", "F", "HR"]
rated = surv.filter(pl.col("Rating").is_in(RATINGS))

curves = {}
for r in RATINGS:
    m, n, d, _ = life_table(rated.filter(pl.col("Rating") == r))
    curves[r] = kaplan_meier(n, d)

print(pl.DataFrame({
    "Rating": RATINGS,
    "loans": [rated.filter(pl.col("Rating") == r).height for r in RATINGS],
    "PD_12": [round(1 - curves[r][1][11], 4) for r in RATINGS],
    "PD_60": [round(1 - curves[r][1][59], 4) for r in RATINGS],
}))

```

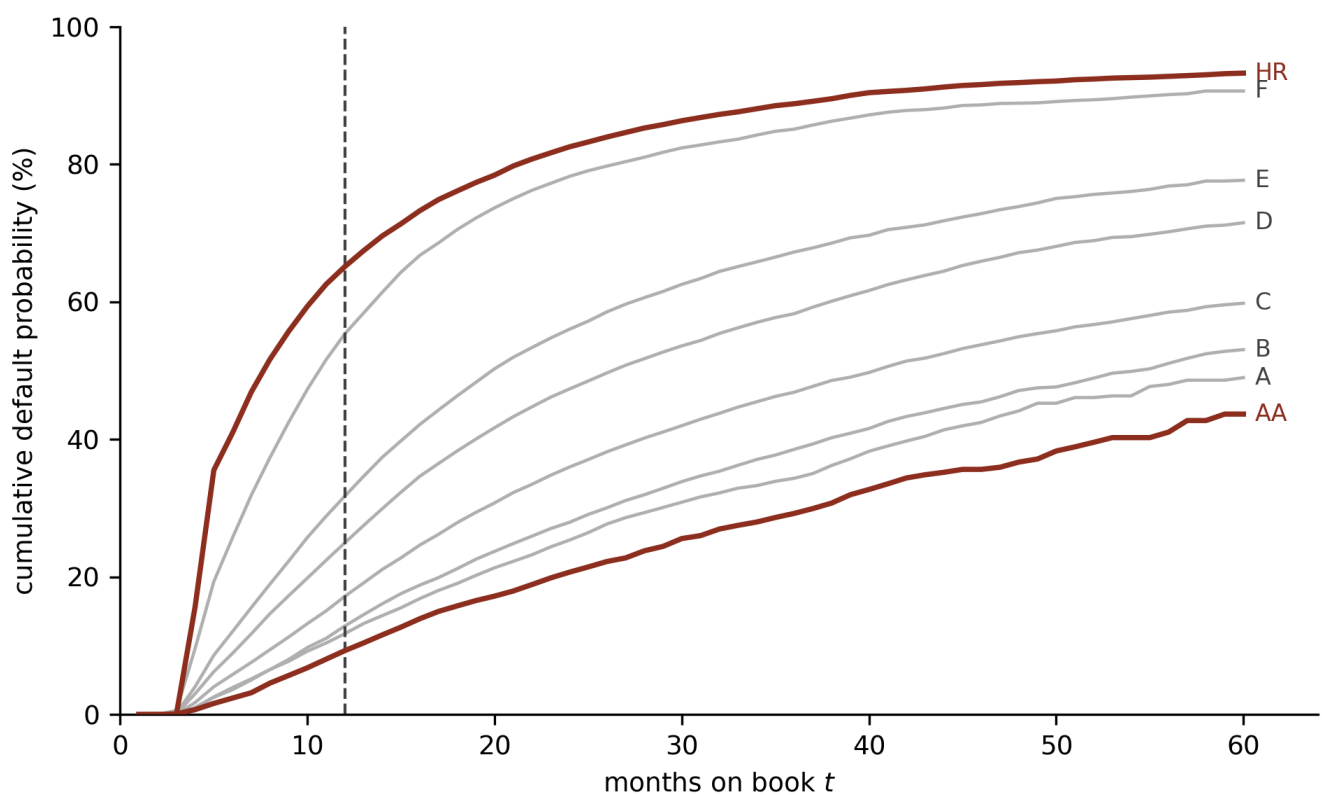
shape: (8, 4)

Rating	loans	PD_12	PD_60
---	---	---	---
str	i64	f64	f64
AA	6582	0.0929	0.4368
A	7551	0.1175	0.4898
B	22589	0.1288	0.5306
C	29766	0.1719	0.598
D	36784	0.2498	0.7151
E	33067	0.3176	0.7768
F	25332	0.5536	0.9067
HR	14831	0.6514	0.9327

```

fig, ax = plt.subplots(figsize=(6.5, 4))
for r in RATINGS:
    S = curves[r][1]
    key = r in ("AA", "HR")
    ax.plot(months, (1 - S) * 100, color=0X if key else GREY,
            lw=1.8 if key else 1.1, zorder=3 if key else 2)
    ax.annotate(r, xy=(60.6, (1 - S[-1]) * 100), fontsize=8, va="center",
               color=0X if key else RULE)
ax.axvline(12, color=RULE, ls="--", lw=1)
ax.set_xlim(0, 64); ax.set_ylim(0, 100)
ax.set_xlabel("months on book $t$"); ax.set_ylabel("cumulative default probability (%)")
plt.tight_layout(); plt.show()

```



Kaplan-Meier default curves by Bondora listing grade, on the 176,502 loans carrying a grade. The eight curves are strictly ordered at every month over the full sixty and never cross, so the grade separates risk over the whole horizon rather than only at the twelve-month point a scorecard would be cut for. Bronze marks the two extreme grades.

```

observed = np.zeros(len(RATINGS)); expected = np.zeros(len(RATINGS))
t_by = [rated.filter(pl.col("Rating") == r)["t_obs_m"].to_numpy().clip(max=HORIZON
    + 1)
    for r in RATINGS]
k_by = [rated.filter(pl.col("Rating") == r)["exit_kind"].to_numpy() for r in
    RATINGS]
for m in range(1, HORIZON + 1):
    n_j = np.array([(t >= m).sum() for t in t_by], dtype=float)
    d_j = np.array([(t == m) & (k == "default").sum() for t, k in zip(t_by,
        k_by)],
        dtype=float)
    if d_j.sum() == 0 or n_j.sum() == 0:
        continue
    observed += d_j
    expected += d_j.sum() * n_j / n_j.sum()
chi2 = (((observed - expected) ** 2) / expected).sum()
dof = len(RATINGS) - 1
print(f"log-rank chi2 = {chi2:,.1f} on {dof} df, p = {stats.chi2.sf(chi2,
    dof):.3g}")

```

```
log-rank chi2 = 29,022.7 on 7 df, p = 0
```

The log-rank test compares the defaults observed in each grade against those expected if every grade shared one hazard, accumulating the comparison month by month over the risk sets. At $\chi^2 = 29,022.7$ on 7 degrees of freedom the null of a common hazard is not remotely tenable, which is the formal counterpart of eight curves that never touch.

6 A discrete-time hazard model is a logistic regression

Here is the result that makes survival analysis immediately available to a lender. Stratifying by one grade at a time does not scale, so the hazard needs covariates, and the way to get them requires no new estimation machinery at all.

Restructure the data so that each loan contributes **one row per month it was at risk**, carrying a binary response equal to one in the month it defaulted and zero otherwise. Botha and Verster (2025) call the resulting response an event history indicator e_{it} , following Singer and Willett (1993), and note that it produces the vector $(0, 0, \dots, 1)$ for a defaulted loan and $(0, 0, \dots, 0)$ for a

censored one. This layout is the **person-period expansion**, and on it the conditional default probability $q_{i,t}$ is simply the mean of a binary response. Any method that maximises a binomial likelihood therefore fits it.

```
def person_period(df, horizon=HORIZON):
    """One row per loan-month at risk, with the event history indicator."""
    return (
        df.with_columns(n_periods=pl.min_horizontal(pl.col("t_obs_m"),
            pl.lit(horizon)))
        .with_columns(t=pl.int_ranges(1, pl.col("n_periods") + 1))
        .explode("t")
        .with_columns(
            y=((pl.col("t") == pl.col("t_obs_m"))
                & (pl.col("exit_kind") == "default")).cast(pl.Int8)
        )
    )

pp = person_period(surv)
print(f"{surv.height:,} loans become {pp.height:,} loan-months "
      f"({pp.height / surv.height:.1f} per loan on average)")
print(f"events in the expansion: {pp['y'].sum():,}")

grouped = pp.group_by("t").agg(d=pl.col("y").sum(), n=pl.len()).sort("t")
print("\nexansion hazard equals the life-table hazard:",
      np.allclose((grouped["d"] / grouped["n"]).to_numpy(), q_t))
```

```
179,235 loans become 2,672,193 loan-months (14.9 per loan on average)
events in the expansion: 71,208
```

```
expansion hazard equals the life-table hazard: True
```

The expansion reproduces the life-table hazards exactly, which is the check that it was built correctly. Note that it is a genuine reshaping rather than a resampling: the 71,208 events are the defaults occurring within sixty months, each appearing once.

Botha and Verster (2025, Eq. 14), following Singer and Willett (1993), then specify the discrete-time hazard model as a logistic regression,

$$h(t \mid \mathbf{E}_i, \mathbf{x}_i, \mathbf{x}_i(t), \mathbf{x}(t)) = \text{logit}^{-1}(\boldsymbol{\alpha}^\top \mathbf{E}_i + \boldsymbol{\beta}^\top \mathbf{x}_i + \boldsymbol{\gamma}^\top \mathbf{x}_i(t) + \boldsymbol{\delta}^\top \mathbf{x}(t)),$$

where $\mathbf{E}_i$ holds indicator variables flagging the period $t \in \{1, \dots, m\}$. Those period indicators together with their coefficients $\boldsymbol{\alpha}$ form the **baseline hazard**, and consequently no stand-alone intercept exists. The remaining three blocks are time-fixed covariates measured at the start of the

spell, spell-level time-varying covariates, and portfolio-level time-varying covariates such as macroeconomic factors.

Only the first two blocks can be fitted here, and the reason is the one lecture 1 established.

Bondora records a single snapshot, so no covariate can vary with t and the terms γ and δ have nothing to estimate from. The macroeconomic block $x(t)$ is exactly what a point-in-time PD needs, and its absence here is the collinearity of $u = g + t$ reappearing in a new guise.

6.1 The baseline hazard alone

Before adding covariates, fit the baseline on its own. A model carrying one indicator per month is saturated in t , so it must return the life-table hazards and nothing else. Grouping the expansion to one row per month first makes the fit instant, and lecture 2 established that the grouped binomial and individual Bernoulli likelihoods agree.

```
import statsmodels.api as sm
import statsmodels.formula.api as smf

g = grouped.to_pandas()
g["period"] = g["t"].astype(str)
saturated = smf.glm("d + I(n - d) ~ C(period) - 1", data=g,
                  family=sm.families.Binomial()).fit()
q_fit = saturated.predict(g).to_numpy()
print("saturated fit reproduces the life table:",
      np.allclose(q_fit, q_t, atol=1e-8),
      f"(max abs difference {np.abs(q_fit - q_t).max():.2e})")
print(f"PD_60 from the fitted hazards : {1 - np.prod(1 - q_fit):.6f}")
print(f"PD_60 from Kaplan-Meier      : {1 - S_t[-1]:.6f}")
```

```
saturated fit reproduces the life table: True (max abs difference 7.19e-15)
PD_60 from the fitted hazards : 0.716798
PD_60 from Kaplan-Meier      : 0.716798
```

The two agree to fourteen decimal places. Hence Kaplan-Meier and a period-indicator logistic regression are the same estimator wearing different clothes.

That fit also emits a perfect-separation warning, suppressed here by the document's `warning: false` setting but real. Month one carries zero defaults, so its coefficient wants to run to minus infinity and is not identified. The fix is to band the sparse early months, which the covariate model does.

6.2 Adding covariates

The covariates are those of lecture 1's model GLM3, namely age class, log-income and country, so the comparison that follows is like for like. The baseline is banded into nine intervals, which removes the separation and keeps the shape.

```

BANDS = [0, 3, 6, 9, 12, 18, 24, 36, 48, 60]
BAND_LABELS = [f"{a + 1}-{b}" for a, b in zip(BANDS[:-1], BANDS[1:])]
AGE_CUTS = [18, 20, 25, 30, 40, 50, 60, 70, 101]
AGE_LABELS = ["18-20", "21-25", "26-30", "31-40", "41-50", "51-60", "61-70",
              "71+"]

def add_design(frame):
    frame = frame.copy()
    frame["AgeClass"] = pd.cut(frame["Age"], AGE_CUTS, labels=AGE_LABELS,
                               include_lowest=True)

    return frame

base = (surv.filter(pl.col("Age").is_not_null() & (pl.col("IncomeTotal") > 0))
        .with_columns(logIncome=pl.col("IncomeTotal").log()))
print(f"loans after lecture 1's cleaning filter: {base.height:,}")

pp_cov = person_period(base).select(["y", "t", "Age", "logIncome", "Country"])
df = add_design(pp_cov.to_pandas())
df["period"] = pd.cut(df["t"], BANDS, labels=BAND_LABELS)

dth = smf.glm("y ~ C(period) + AgeClass + logIncome + Country - 1",
              data=df, family=sm.families.Binomial()).fit()
print(f"\nfitted on {int(dth.nobs):,} loan-months, deviance {dth.deviance:,.1f}")
print("\nbaseline hazard coefficients (alpha):")
print(dth.params.filter(like="period").round(4).to_string())
print(f"\nlogIncome coefficient: {dth.params['logIncome']:+.4f}")

```

loans after lecture 1's cleaning filter: 179,171

fitted on 2,671,330 loan-months, deviance 593,594.4

baseline hazard coefficients (alpha):

C(period)[1-3]	-7.2333
C(period)[4-6]	-2.6107
C(period)[7-9]	-2.7803
C(period)[10-12]	-2.7631
C(period)[13-18]	-2.8466
C(period)[19-24]	-2.9903
C(period)[25-36]	-3.1997
C(period)[37-48]	-3.3336
C(period)[49-60]	-3.7206

logIncome coefficient: -0.0669

Two features of that output tie the lecture back into the series. The baseline coefficients trace the lifecycle directly: months one to three sit at -7.23 on the logit scale, effectively zero hazard, then the band four to six jumps to -2.61 and the coefficients decline monotonically thereafter to

−3.72 over months 49 to 60. Furthermore, the log-income coefficient is **negative** at −0.0669. Lecture 1 found the marginal income effect positive and showed the sign was a composition artefact that reversed once country entered the model. Here the same reversal holds in a hazard model over sixty months, so the Simpson's paradox lecture 1 diagnosed was a property of the portfolio rather than of the twelve-month window.

6.3 The term structure

The payoff is that one fitted model now delivers a PD at every horizon. Chaining the predicted hazards per loan and averaging over the portfolio gives the term structure of default risk.

```
loans = add_design(base.select(["Age", "logIncome", "Country"]).to_pandas())
S_i = np.ones(len(loans))
pd_curve = np.empty(HORIZON)
for k in range(1, HORIZON + 1):
    grid = loans.copy()
    grid["period"] = pd.cut([k] * len(loans), BANDS, labels=BAND_LABELS)
    S_i = S_i * (1 - dth.predict(grid).to_numpy())
    pd_curve[k - 1] = 1 - S_i.mean()

print(f"model PD_12 {pd_curve[11]:.4f}    Kaplan-Meier PD_12 {1 - S_t[11]:.4f}")
print(f"model PD_60 {pd_curve[59]:.4f}    Kaplan-Meier PD_60 {1 - S_t[59]:.4f}")
```

```
model PD_12 0.3077    Kaplan-Meier PD_12 0.3076
model PD_60 0.7120    Kaplan-Meier PD_60 0.7168
```

```

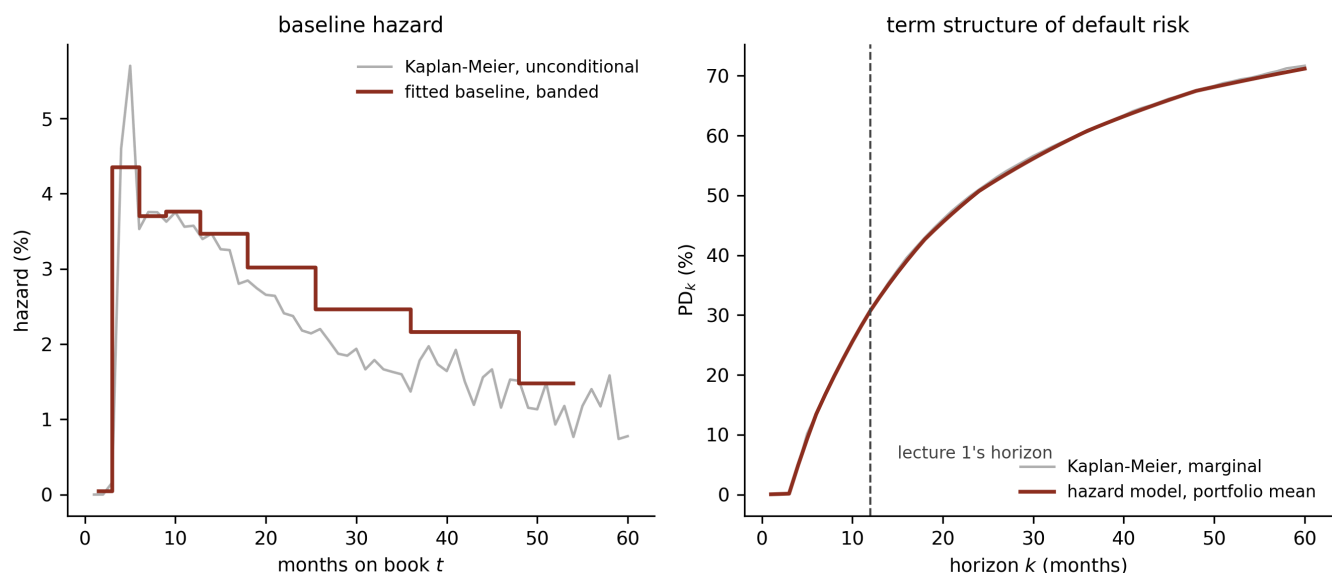
band_mid = [(a + b) / 2 for a, b in zip(BANDS[:-1], BANDS[1:])]
band_haz = [1 / (1 + np.exp(-dth.params[f"C(period)[{lab}]"
    - dth.params["logIncome"] *
    loans["logIncome"].mean()))
    for lab in BAND_LABELS]

fig, axs = plt.subplots(1, 2)

ax = axs[0]
ax.plot(months, q_t * 100, color=GREY, lw=1.2, label="Kaplan-Meier,
    unconditional")
ax.step(band_mid, np.array(band_haz) * 100, where="mid", color=0X, lw=1.8,
    label="fitted baseline, banded")
ax.set_xlabel("months on book $t$"); ax.set_ylabel("hazard (%)")
ax.legend(frameon=False, fontsize=8)
ax.set_title("baseline hazard", fontsize=10)

ax = axs[1]
ax.plot(months, (1 - S_t) * 100, color=GREY, lw=1.2, label="Kaplan-Meier,
    marginal")
ax.plot(months, pd_curve * 100, color=0X, lw=1.8, label="hazard model, portfolio
    mean")
ax.axvline(12, color=RULE, ls="--", lw=1)
ax.annotate("lecture 1's horizon", xy=(12, 6), xytext=(15, 6), fontsize=8,
    color=RULE)
ax.set_xlabel("horizon $k$ (months)"); ax.set_ylabel("$\\mathrm{PD}_k$ (%)")
ax.legend(frameon=False, fontsize=8, loc="lower right")
ax.set_title("term structure of default risk", fontsize=10)
plt.tight_layout(); plt.show()

```



The term structure of default risk from the fitted discrete-time hazard model, averaged over the 179,171 loans in the design. (lhs): the fitted baseline hazard by band, as a hazard at the reference profile, against the unconditional Kaplan-Meier hazard. (rhs): the model's cumulative PD by horizon against the marginal Kaplan-Meier curve, which it tracks closely at the short end (30.77 against 30.76 per cent at twelve months) and slightly below at the long end (71.20 against 71.68 at sixty). Both panels treat early repayment as censoring, which the next section shows overstates the level.

7 What treating repayment as censoring costs

Every estimate above treated a settled loan as censored, meaning it was removed from the risk set with no event recorded, exactly as an administratively censored loan is. Botha and Verster (2025) name this the **latent risks approach**, after Putter et al. (2007), and it is the default treatment in most survival software. Nevertheless, it carries an assumption that is false in a loan book.

Censoring is legitimate when it is independent of the event, i.e. when a loan leaving the risk set tells you nothing about its default risk. Early repayment fails that test in both directions. A borrower who repays early is disproportionately a borrower who would not have defaulted, so removing them leaves a risk set enriched in weak credits and drives the estimated hazard up. Consequently early repayment is a **competing risk** rather than censoring, and the estimator that respects the distinction is the cumulative incidence function, estimated by Aalen-Johansen.

The two estimators differ in one place. Kaplan-Meier weights each month's hazard by survival with respect to default alone, whereas the cumulative incidence function weights it by survival with respect to **any** exit:

$$\widehat{\text{CIF}}_{\text{def}}(k) = \sum_{t=1}^k \widehat{S}_{\text{any}}(t-1) \hat{q}_{\text{def},t}, \quad \widehat{S}_{\text{any}}(t) = \prod_{s=1}^t (1 - \hat{q}_{\text{def},s} - \hat{q}_{\text{set},s}).$$

```
def cumulative_incidence(df, horizon=HORIZON):
    """Aalen-Johansen CIFs for default and settlement, plus overall survival."""
    t = df["t_obs_m"].to_numpy().clip(max=horizon + 1)
    kind = df["exit_kind"].to_numpy()
    m = np.arange(1, horizon + 1)
    n = np.array([(t >= x).sum() for x in m], dtype=float)
    d = np.array([(t == x) & (kind == "default").sum() for x in m], dtype=float)
    s = np.array([(t == x) & (kind == "settled").sum() for x in m], dtype=float)
    q_d, q_s = d / n, s / n
    S_any = np.cumprod(1 - q_d - q_s)
    S_lag = np.concatenate([[1.0], S_any[:-1]])
    return np.cumsum(S_lag * q_d), np.cumsum(S_lag * q_s), S_any

cif_def, cif_set, S_any = cumulative_incidence(surv)
km_pd = 1 - S_t

print(f"identity check, CIF_def + CIF_set + S_any = {cif_def[-1] + cif_set[-1] + S_any[-1]:.12f}")
print()
for k in (12, 60):
    print(f"k = {k:2d}    Kaplan-Meier {km_pd[k-1]:7.2%}    "
          f"cumulative incidence {cif_def[k-1]:7.2%}    "
          f"overstatement {100 * (km_pd[k-1] - cif_def[k-1]):+.2f}pp")
```

```
identity check, CIF_def + CIF_set + S_any = 1.000000000000
```

```
k = 12    Kaplan-Meier  30.76%    cumulative incidence  27.72%    overstatement
+3.05pp
k = 60    Kaplan-Meier  71.68%    cumulative incidence  54.69%    overstatement
+16.99pp
```

The three quantities sum to one to twelve decimal places, which is the check that the decomposition is complete: every loan has defaulted, settled or survived. Over twelve months the latent risks treatment overstates the PD by 3.05 percentage points, and over sixty by 16.99, taking a lifetime PD of 54.69 per cent up to 71.68.

7.1 Lecture 1 as the test case

The size of the error can be asserted, or it can be tested against a sample where the truth is directly observable, and lecture 1 supplies one. On its seasoned sample every loan has a complete

twelve-month window by construction, so no loan is administratively censored inside the horizon, and the fraction defaulting within twelve months is simply counted, with nothing left to estimate. That fraction was 28.95 per cent.

```
pd_table = pl.read_parquet("../data/bondora_pd.parquet").select(["LoanId",
    "default_12m"])
seasoned = surv.join(pd_table, on="LoanId", how="inner")

m12, n12, d12, _ = life_table(seasoned, 12)
_, S12, _ = kaplan_meier(n12, d12)
cif12, _, _ = cumulative_incidence(seasoned, 12)
truth = seasoned["default_12m"].mean()

censored_inside = seasoned.filter(
    (pl.col("exit_kind") == "censored") & (pl.col("t_obs_m") <= 12)).height
settled_inside = seasoned.filter(
    (pl.col("exit_kind") == "settled") & (pl.col("t_obs_m") <= 12)).height
print(f"seasoned loans {seasoned.height:,}; censored inside 12 months "
    f"{censored_inside:,}; settled inside 12 months {settled_inside:,}")
print()
print(f"observed truth, complete windows    {truth:.6f}")
print(f"cumulative incidence at 12 months    {cif12[-1]:.6f}    "
    f"error {100 * (cif12[-1] - truth):+.4f}pp")
print(f"Kaplan-Meier at 12 months            {1 - S12[-1]:.6f}    "
    f"error {100 * ((1 - S12[-1]) - truth):+.4f}pp")
```

```
seasoned loans 148,733; censored inside 12 months 0; settled inside 12 months
21,393
```

```
observed truth, complete windows    0.289492
cumulative incidence at 12 months    0.289492    error +0.0000pp
Kaplan-Meier at 12 months            0.320622    error +3.1130pp
```

The cumulative incidence function recovers lecture 1's default rate exactly, while Kaplan-Meier misses it by 3.11 percentage points. The agreement is an identity rather than a coincidence. Where no loan is censored inside the horizon, the cumulative incidence telescopes,

$$\sum_{t=1}^k \widehat{S}_{\text{any}}(t-1) \cdot \frac{d_t}{n_t} = \sum_{t=1}^k \frac{n_t}{n_1} \cdot \frac{d_t}{n_t} = \frac{1}{n_1} \sum_{t=1}^k d_t,$$

which is the observed proportion. Hence the fixed-horizon PD of lecture 1 and the competing-risks survival estimate are the same quantity computed two ways, and the survival machinery generalises lecture 1 rather than contradicting it. What Kaplan-Meier estimates on this sample is

a different quantity: the PD that would obtain if early repayment were somehow abolished while default risk stayed put.

The correction, and the regression model that carries it (the Fine-Gray subdistribution hazard), belong to lecture S2. Note carefully that the term structure fitted above inherits this bias, since it too treats settlement as censoring, so its levels should be read as an upper bound.

8 Takeaways

Survival analysis reaches credit risk through a short bridge, and this lecture crossed it in four steps.

1. A fixed-horizon PD and a discrete hazard are the same information organised differently. Setting $k = 12$ in $PD_k = 1 - \prod_{t \leq k} (1 - q_{i,t})$ recovers lecture 1 exactly, and every other horizon comes free.
2. A lifetime PD is a product of survival factors. Summing the Bondora hazards over sixty months returns 124.5 per cent against a correct 71.7, so the error is disqualifying rather than approximate.
3. A discrete-time hazard model is a logistic regression on a person-period expansion. A period-indicator model reproduces Kaplan-Meier to fourteen decimal places, and adding covariates needs no machinery a lender does not already own.
4. Early repayment is a competing risk. Treating it as censoring overstates the sixty-month PD by 17.0 percentage points here, and the cumulative incidence function recovers lecture 1's observed rate exactly where Kaplan-Meier misses it by 3.11.

One limitation runs through all of it. The extract holds a single snapshot, so no covariate varies with time and the portfolio-level block $x(t)$ of the Botha and Verster specification cannot be fitted. That block is where a point-in-time PD comes from, which returns the lecture to the identity $u = g + t$ and to `data/amex_panel.parquet`, where the question can actually be asked.

Lecture S2 takes up the translation between the actuarial and statistical traditions, the central exposed to risk against the person-period expansion, and the Fine-Gray correction this lecture measured and deferred.

Copyright and attribution

This lecture answers no lecture of the summer school **Deep Learning for Actuarial Modeling** and is original to this repository, written because credit lecture 1 opened a door it did not walk through. It inherits that lecture's notation, which in turn adapts Scognamiglio, Maggi and Wüthrich (2026). The discrete-time hazard identities, the R_{ij} resolution encoding and the model specification follow Botha and Verster (2025); the Bondora loan book is public.

References

- Aalen, O.O. and Johansen, S. (1978). *An empirical transition matrix for non-homogeneous Markov chains based on censored observations*. Scandinavian Journal of Statistics.
- Bondora AS. *Loan dataset (public reports)*. <https://www.bondora.com/en/public-reports>
- Botha, A. and Verster, T. (2025). *Approaches for modelling the term-structure of default risk under IFRS 9: a tutorial using discrete-time survival analysis*.
- Breeden, J.L. (2016). *Incorporating lifecycle and environment in loan-level forecasts and stress tests*. European Journal of Operational Research.
- Fine, J.P. and Gray, R.J. (1999). *A proportional hazards model for the subdistribution of a competing risk*. Journal of the American Statistical Association.
- Greenwood, M. (1926). *The natural duration of cancer*. Reports on Public Health and Medical Subjects.
- International Accounting Standards Board (2014). *IFRS 9: Financial Instruments*, section 5.5: impairment.
- Kaplan, E.L. and Meier, P. (1958). *Nonparametric estimation from incomplete observations*. Journal of the American Statistical Association.
- Putter, H., Fiocco, M. and Geskus, R.B. (2007). *Tutorial in biostatistics: competing risks and multi-state models*. Statistics in Medicine.
- Singer, J.D. and Willett, J.B. (1993). *It's about time: using discrete-time survival analysis to study duration and the timing of events*. Journal of Educational Statistics.
- Scognamiglio, S., Maggi, M. and Wüthrich, M.V. (2026). *Deep Learning for Actuarial Modeling*, SSRN 5162304.